

Hackathon Challenge 1

CS60055–Ubiquitous Computing

Indian Institute of Technology Kharagpur

Department of Computer Science and Engineering

Submitted by

Deepesh Singh (22CS30020)

Saptarshi Sarkar (22CS30047)

Animesh Kumar (22CS30009)

September 11, 2026

Contents

1	The Design of the Solution Along with the Salient Ideas	1
1.1	Overview	1
1.2	The Core Idea: Separate ‘Deciding’ from ‘Explaining’	1
1.3	System Architecture	1
1.4	Role of Ollama and the Local Language Model	2
1.5	The Four Question Types Supported	3
2	How the Solution Was Implemented	4
2.1	Dataset	4
2.2	Feature Engineering	4
2.3	Recognition Models	5
2.4	Aggregation and Evidence	5
2.5	Interface Layer and Output Format	6
2.6	Efficiency Measurement	6
3	Test Results	7
3.1	Evaluation Setup	7
3.2	Recognition Accuracy	7
3.3	Question-Answering Accuracy	8
3.4	Robustness	9
3.5	Efficiency Versus Accuracy	10
4	Challenges Faced and How They Were Approached	12
5	Mandatory Disclosure if AI Tools Were Used	13

Chapter 1

The Design of the Solution Along with the Salient Ideas

1.1 Overview

Ask the Sensors is a question-answering system that answers natural-language questions about a person’s physical activity using only accelerometer and gyroscope data. For example, a user can ask “How long was the user walking?” or “Did the user lie down for a prolonged period?” and the system produces a structured answer that is tied back to the actual sensor evidence that supports it.

1.2 The Core Idea: Separate ‘Deciding’ from ‘Explaining’

Large language models generate fluent text but are not reliable sources of facts, which is critical for systems involving health or daily-routine reasoning.

Therefore, every number, timestamp, and duration is computed by deterministic code, never by the language model. A local model served through **Ollama** only (a) interprets the user’s natural-language query and (b) converts the computed evidence into fluent text. It never receives raw sensor values or computes the answer. If Ollama is unavailable, the system remains functional using more rigid, template-based responses.

1.3 System Architecture

The system is organized as four layers, matching the pipeline suggested in the challenge brief. Figure 1.1 shows how data flows through them.

Preprocessing layer. Cleans per-minute sensor data by removing frequently missing features, filling remaining gaps with per-user medians, detecting large time gaps to avoid incorrect interval stitching, and adding derived features such as gyroscope-to-accelerometer magnitude ratios. For raw accelerometer/gyroscope streams, it also resamples data to a uniform 25 Hz grid before feature extraction.

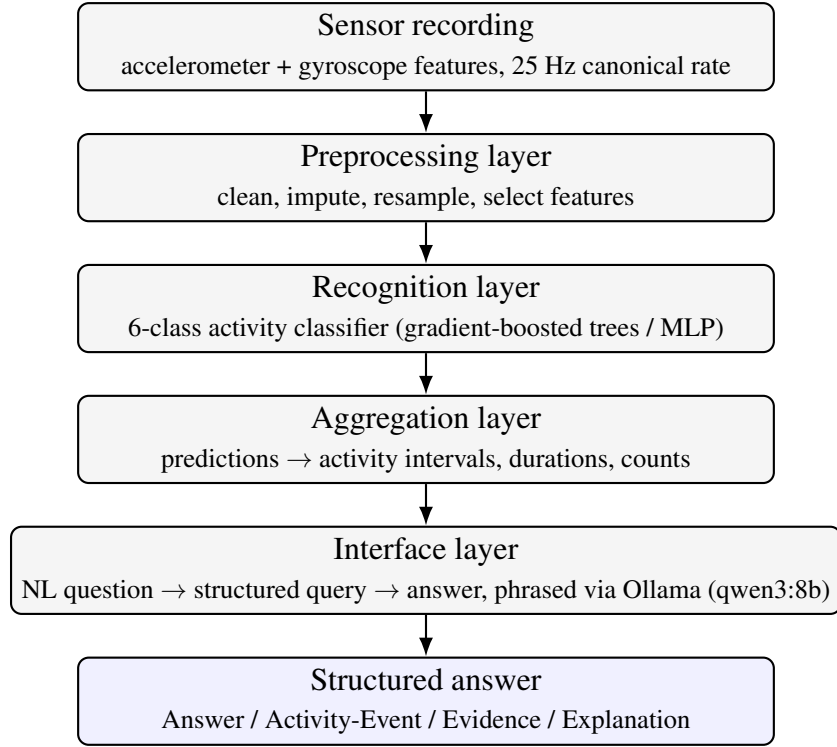


Figure 1.1: Overall system architecture.

Recognition layer. A supervised classifier, not a language model, predicts one of six activity classes: Lying down, Sitting, Standing in place, Walking, Running, and Bicycling. Three classifier variants are compared in Section 2.

Aggregation layer. Converts per-minute predictions into activity intervals. Durations, counts, and comparisons are computed directly from this timeline using deterministic arithmetic, without a language model.

Interface layer. The only layer that uses a language model.

1.4 Role of Ollama and the Local Language Model

Ollama serves the local `qwen3:8b` model through an HTTP API and is restricted to the interface layer:

1. **Understanding the question.** A rule-based parser handles most queries. If it cannot determine the intent confidently, the language model fills fixed structured fields using only known activities and operations.
2. **Phrasing the explanation.** After deterministic computation, the model converts the evidence description into one or two fluent sentences and is instructed not to invent numbers, timestamps, or signal details.

Ollama keeps processing local, avoiding external transmission of user activity data. If unavailable, intent parsing falls back to rules and keyword matching, while explanations use fixed

templates. The Answer, Activity/Event, and Evidence fields remain unaffected because they do not depend on the language model.

1.5 The Four Question Types Supported

1. **Activity identification** – identifies or verifies an activity at a given time.
2. **Temporal and quantitative reasoning** – answers duration, count, and comparison questions.
3. **Evidence grounding** – identifies when an activity started and returns the supporting time interval.
4. **Open-world reasoning** – handles semantic judgments, such as prolonged lying down using a fixed duration threshold or identifying qualitatively described activities.

Chapter 2

How the Solution Was Implemented

2.1 Dataset

The system was built and evaluated on the **ExtraSensory** dataset, a public, real-world, in-the-wild activity-sensing dataset in which around 60 users carried a smartphone and reported their own activity labels throughout normal daily life. Only two parts of the public release were used:

- The primary features-and-labels file, which contains, for every recorded minute, roughly 225 already-computed accelerometer and gyroscope features together with the corresponding activity labels.
- The official 5-fold, user-independent cross-validation partition published with the dataset, so that no single user’s data appears in both the training and the test set within the same fold, and results are comparable to prior published work on this dataset.

The larger raw accelerometer and gyroscope archives from ExtraSensory were not used because the primary file already provides extracted features at the same one-minute granularity required by the project. Reprocessing the raw waveforms would require significantly more storage and time without improving what the model can learn.

Six activity classes had sufficient real, labelled examples: lying down, sitting, standing in place, walking, running, and bicycling. Only accelerometer- and gyroscope-derived features were used, while features from other sensors, such as location and audio, were excluded to maintain the sensor-only wearable setting.

2.2 Feature Engineering

Two preprocessing decisions were made based on direct data inspection:

- **Orientation-dependent features were excluded.** Raw 3-axis sensor features were removed because uncontrolled phone orientation could be confused with activity. In one user’s data, a feature was bimodal within ‘sitting’ examples, with one mode overlapping ‘lying down.’ Therefore, twelve such features were excluded in favor of orientation-invariant, magnitude-based features.

- **Four derived features were added.** These included gyroscope-to-accelerometer magnitude ratios (mean and standard deviation), the ratio of high- to low-frequency accelerometer energy, and the interquartile range of accelerometer magnitude. These features help distinguish walking, running, and bicycling, which can have similar motion intensity but different motion characteristics.

Missing values were filled using each user’s median for that feature, with the global median used as a fallback. Large gaps between consecutive recordings were detected to prevent aggregation from incorrectly spanning periods without sensor data.

2.3 Recognition Models

Three classifiers were trained and compared using **scikit-learn**:

- **Full model.** A gradient-boosted decision tree ensemble trained across all classes, used as the main model.
- **Hierarchical model.** A two-stage classifier that first separates ‘stationary’ and ‘moving’ activities, then identifies the specific activity within each group. This reflects the confusion matrix pattern, where errors occur mainly within these groups rather than between them.
- **Compressed model.** A small multi-layer perceptron with one 32-unit hidden layer, with weights quantized to 16-bit floating point after training to evaluate the accuracy-resource trade-off.

Because some classes are much rarer than others, **SMOTE** from `imbalanced-learn` was used to generate additional minority-class samples, along with class-balanced weighting for the gradient-boosted models. This reduces bias toward common classes and makes macro-F1 and balanced accuracy more meaningful. All randomness in data splitting, SMOTE sampling, and model initialization was controlled by a single seed, ensuring reproducible results for each configuration.

2.4 Aggregation and Evidence

The recognition layer’s per-minute predictions are converted into a timeline of activity intervals by grouping consecutive same-activity predictions together, respecting the recording-gap detection described above. All duration, count, and comparison questions are then answered by simple, deterministic operations over this timeline. For each answer, the system also builds a short, automatically generated natural-language description of the supporting signal, which is what the language model is later allowed to phrase into a full sentence, and nothing more.

2.5 Interface Layer and Output Format

The interface layer, described in Section 1.4, ties the above pieces together: it interprets the incoming question (rule-based first, with Ollama/qwen3:8b as a constrained fallback), decides which deterministic timeline operation answers it, and renders the result in the exact five-field format required by the brief (*Answer*, *Activity/Event*, *Evidence* with timestamps/modality/channel, and *Explanation*). For identification-style questions, the brief explicitly allows the Evidence fields to be reported as N/A, which the implementation follows.

2.6 Efficiency Measurement

For each of the three recognition models, the following were measured directly: number of parameters, size on disk, peak memory usage during inference, and the average time taken to answer a single query (averaged over 50 repeated timing runs). These measurements were used to build the accuracy-versus-overhead comparison presented in Section 3.

Chapter 3

Test Results

3.1 Evaluation Setup

The system was evaluated using the official 5-fold, user-independent cross-validation split described in Section 2.1, so that reported numbers reflect performance on users the model never saw during training. The **full** gradient-boosted model was used as the headline recognition model for the question-answering accuracy numbers below; a separate comparison across all three recognition-model variants is given in Section 3.4.

3.2 Recognition Accuracy

Table 3.1 and Figure 3.1 show the classifier’s per-class performance on held-out users.

Activity	Precision	Recall	F1
Lying down	0.63	0.38	0.47
Sitting	0.55	0.66	0.60
Standing in place	0.20	0.23	0.21
Walking	0.43	0.52	0.47
Running	0.10	0.15	0.12
Bicycling	0.39	0.60	0.47
Macro F1	0.39		
Balanced accuracy	0.42		

Table 3.1: Per-class recognition performance, full model, held-out users.

The results are honest but modest: overall recognition is well above chance (chance for six roughly imbalanced classes would be well under 0.20 macro F1), and sitting, lying down, Walking, and bicycling are all recognized with F1 around 0.47–0.60. However, standing in place and running are recognized poorly (F1 of 0.21 and 0.12, respectively). Figure 3.1 shows that this is not random error: confusion is concentrated almost entirely *within* two natural groups. Lying down / Sitting / Standing (the stationary group) and Walking / Running / Bicycling (the moving

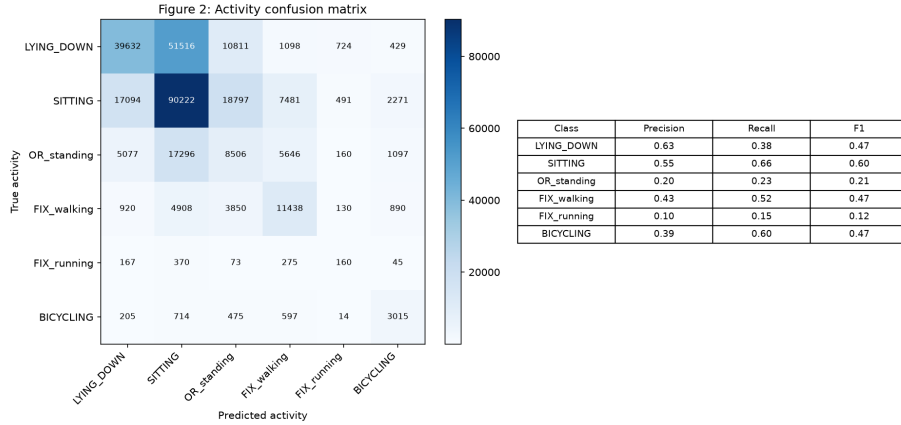
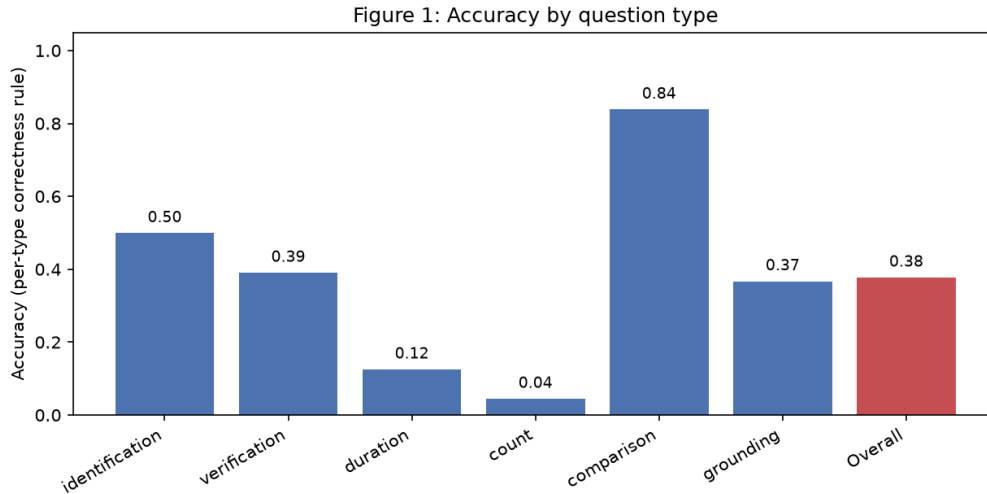


Figure 3.1: Activity confusion matrix, full model, held-out users.

group) with very little confusion *across* the two groups. The single largest source of error is lying down being predicted as sitting.

3.3 Question-Answering Accuracy

Figure 3.2 breaks down end-to-end question-answering accuracy by question type, using the correctness rule appropriate to each type (exact match for identification; positive-class F1 for verification; within-tolerance match for duration; exact match for count; correct larger/smaller judgment for comparison; and answer correctness combined with sufficient timestamp overlap for grounding).



Correctness rule per group -> identification: exact match; verification: F1 on positive class (avg over 7 classes); duration: within 60s abs or 10% rel; count: within +/-1; comparison: exact match on which activity had more total time; grounding: IoU >= 0.3

Figure 3.2: Accuracy by question type.

Performance varies substantially by question type. Comparison questions are answered correctly 84% of the time, since these only require the aggregation layer to get the *relative* ordering of two totals right, which is more forgiving than getting an exact number right. Identification

(50%) and verification (39%) track the underlying recognition accuracy directly, as expected. Evidence-grounding questions are correct 37% of the time at the default overlap threshold. Duration (12%) and count (4%) questions perform the worst: even where the recognized activity is correct, the exact boundary of an activity interval is much harder to get precisely right than its identity, and small per-minute misclassifications accumulate into larger timing errors once they are summed into a duration or a count. Overall, macro-averaged question-answering accuracy across all types is 0.38.

Figure 3.3 shows how both duration and grounding accuracy change as the correctness criterion is loosened: grounding accuracy in particular rises sharply (from 0.37 to over 0.60) as the required timestamp overlap is relaxed, showing that many “incorrect” grounded answers are close to correct rather than wildly wrong.

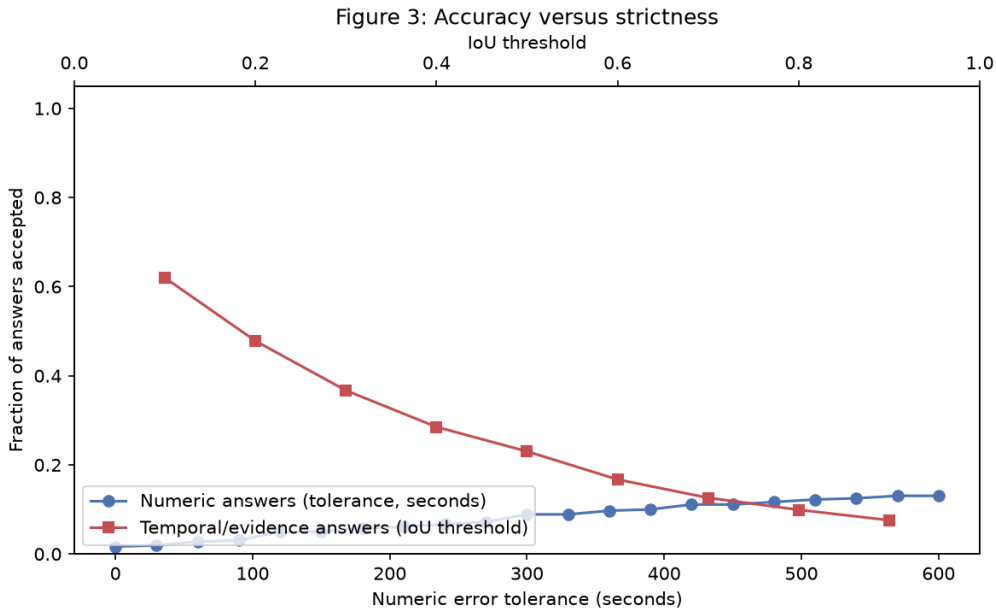


Figure 3.3: Accuracy versus strictness of the correctness criterion.

3.4 Robustness

Figures 3.4 and 3.5 show how question-answering accuracy on a fixed question set changes as the input signal is degraded, first by adding random noise to each feature, then by randomly dropping a fraction of the feature values.

The system is reasonably robust to both kinds of degradation. Accuracy drops modestly from 0.49 to around 0.42–0.44 as soon as any noise is added and then stays roughly flat even as the noise level is increased substantially further. The model does not collapse under heavier noise once it has already adjusted to the presence of some. Accuracy degrades more gradually and predictably under sample dropout, falling from 0.49 at no dropout to about 0.40 at 50% of feature values dropped, without any sharp cliff.

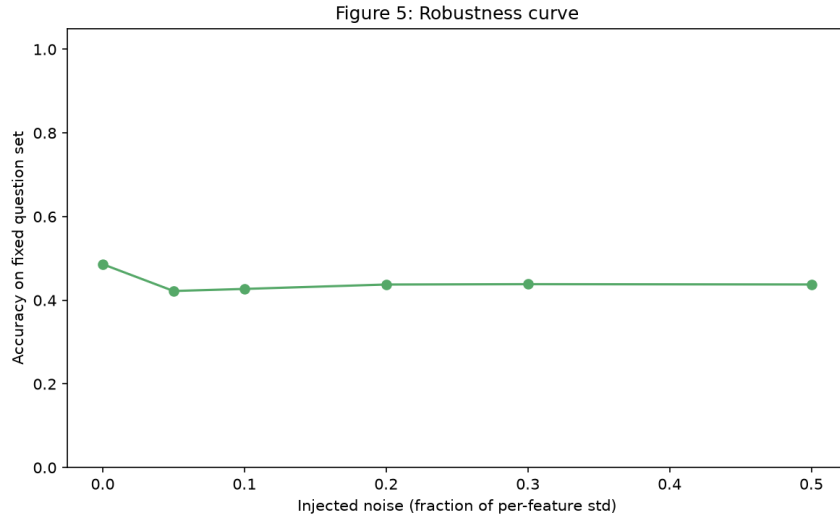


Figure 3.4: Accuracy under added sensor noise.

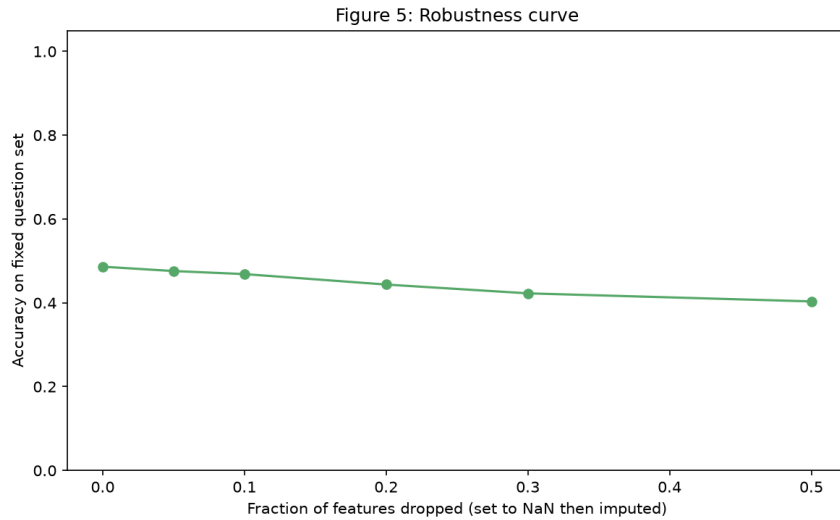


Figure 3.5: Accuracy under dropped sensor samples.

3.5 Efficiency Versus Accuracy

Table 3.2 and Figure 3.6 compare the three recognition-model variants on both accuracy and resource cost, measured on the machine used for development and evaluation of this project.

Model	QA accuracy	Parameters	Size on disk	Latency (mean)	Peak memory
Full	0.499	299,666	16.55 MB	171.6 ms	1.61 MB
Hierarchical	0.482	309,586	17.29 MB	103.9 ms	1.68 MB
Compressed	0.461	1,638	0.05 MB	2.0 ms	1.73 MB

Table 3.2: Accuracy and resource cost by recognition-model variant.

The compressed model is the clearest positive result in this project’s evaluation: it is roughly 300 times smaller on disk and around 85 times faster per query than the full model, while losing

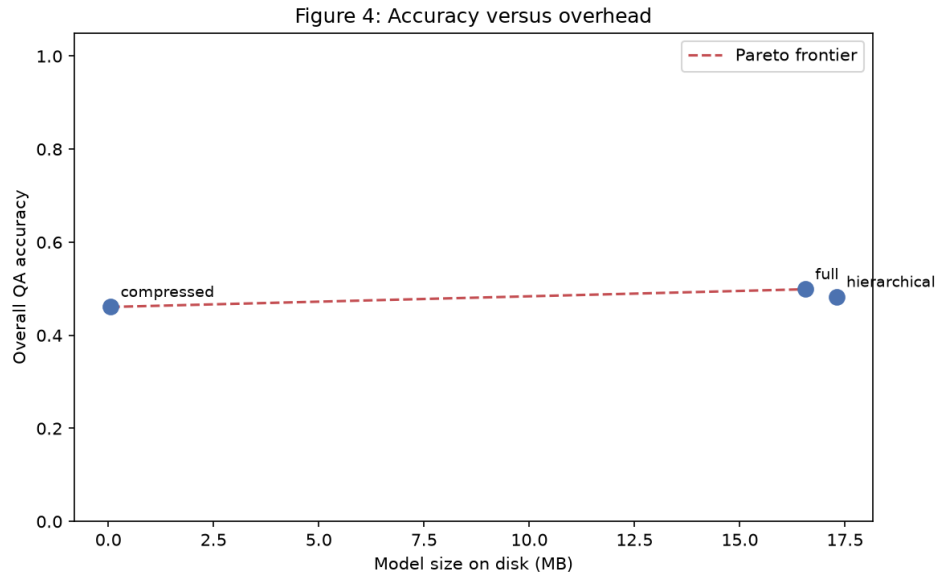


Figure 3.6: Accuracy versus model size on disk.

only about 4 percentage points of overall question-answering accuracy (0.499 to 0.461). This is a genuinely favorable trade-off for any deployment where the model needs to run on constrained hardware, such as a phone or a small embedded board. Interestingly, the hierarchical model, despite being motivated by a real pattern in the confusion matrix, did not outperform the simpler full model in this evaluation (0.482 versus 0.499).

Chapter 4

Challenges Faced and How They Were Approached

This section presents the main limitations observed in the results and how they were addressed.

1. **Overall recognition accuracy is moderate.** Macro-F1 of 0.39 and balanced accuracy of 0.42 show useful performance but leave substantial room for improvement. Class-balanced training using SMOTE and balanced weighting reduced bias toward common classes, while the hierarchical model was designed around the observed confusion pattern. Future work could add class-specific features or test different window sizes.
2. **Standing in place and running are poorly recognized.** Both have F1 below 0.25. Running likely suffers from data scarcity, with under 1% of labelled examples in a typical fold, while SMOTE cannot replace information from real samples. Standing in place appears difficult to distinguish from Sitting and moving activities, possibly due to limited signal in the orientation-invariant features. These weaknesses are reported explicitly rather than hidden by overall accuracy.
3. **Duration and count questions perform poorly.** Their accuracies are only 12% and 4%, respectively, because they depend on correctly identifying activity boundaries, where small per-minute errors accumulate. The strictness analysis (Figure 3.3) shows that even relaxed tolerances keep duration accuracy below 20%, indicating substantial timing errors. A future improvement would be smoothing or interval merging specifically optimized for duration and count accuracy.
4. **The hierarchical model did not outperform the full model.** Although motivated by the observed confusion pattern, splitting training into coarse stationary/moving stages reduces the data available to each classifier, particularly for rare classes. This may outweigh the benefit of the specialized architecture. The negative result is reported honestly as required by the brief.
5. **Evaluation reflects the available labelled data.** One considered activity class, Standing and moving, lacked sufficient real labelled examples and was therefore excluded from the recognition classes, confusion matrix, and Section 3 accuracy results. This limitation is stated explicitly rather than silently changing the class list.

Chapter 5

Mandatory Disclosure if AI Tools Were Used

An AI tool was used in this project as part of the running system itself:

- **Ollama, serving the `qwen3:8b` language model locally**, is used inside the implemented pipeline's interface layer. Its role is strictly limited to (a) parsing a natural-language question into a structured intent when a faster rule-based parser cannot confidently do so, and (b) phrasing the final *Explanation* field of an answer from a deterministic, pre-computed evidence description. It does not compute any answer, timestamp, or duration reported by the system, and the system is designed to fall back to rule-based logic and template text if it is unavailable, as described in Sections 1 and 2.